

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №3
по дисциплине «Вычислительная математика»
Тема: метод бисекции

Студент гр. 4341

Четвертная В.Л.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Целью лабораторной работы являются изучение метода бисекции для поиска корня уравнения с заданной точностью, написание программы, реализующей данный метод, и исследование зависимости числа итераций от точности и обусловленности метода (чувствительности к ошибкам в исходных данных).

Задание.

Требуется найти корень уравнения $f(x)=0$ методом бисекции с заданной точностью Eps , исследовать зависимость числа итераций от точности Eps при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных).

Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения $f(x)=0$ (т.е. найти отрезки $[Left, Right]$, на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).

2) Составить подпрограмму вычисления функции $f(x)$.

3) Составить головную программу, содержащую обращение к подпрограмме $f(x)$, *BISECT*, *Round* и индикацию результатов.

4) Провести вычисления по программе. Построить график зависимости числа итераций от Eps .

5) Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы *Round*, округляющей значения функции с заданной точностью $Delta$.

Вариант 23: $f(x) = \sinh(x) - x + 1$

Отделение корней уравнения.

Рассмотрим функцию $f(x) = \sinh(x) - x + 1$ и её поведение. Она определена и непрерывна на $(-\infty; +\infty)$. Её производная равна:

$$f'(x) = \cosh(x) - 1$$

Производная равна нулю в точке $x=0$, на остальном участке $(-\infty; 0) \cup (0; +\infty)$ положительна. Следовательно 0 является точкой перегиба функции, на всей области определения функция монотонно возрастает.

При $x \rightarrow -\infty: f(x) \rightarrow -\infty$, при $x \rightarrow +\infty: f(x) \rightarrow +\infty$.

Функция непрерывна на $(-\infty; +\infty)$ и меняет знак с $-\infty$ на $+\infty$. По теореме Больцано-Коши корень существует. Так как функция строго возрастает, корень единственный.

Локализуем корень:

$$x = -2: f(-2) = \sinh(-2) + 2 - 1 \approx -2.627 < 0$$

$$x = -1: f(-1) = \sinh(-1) + 1 - 1 \approx 0.825 > 0$$

Аналитические расчёты подтверждаются графически (см. рис.1).

Для метода бисекции возьмём отрезок $[-2; -1]$.

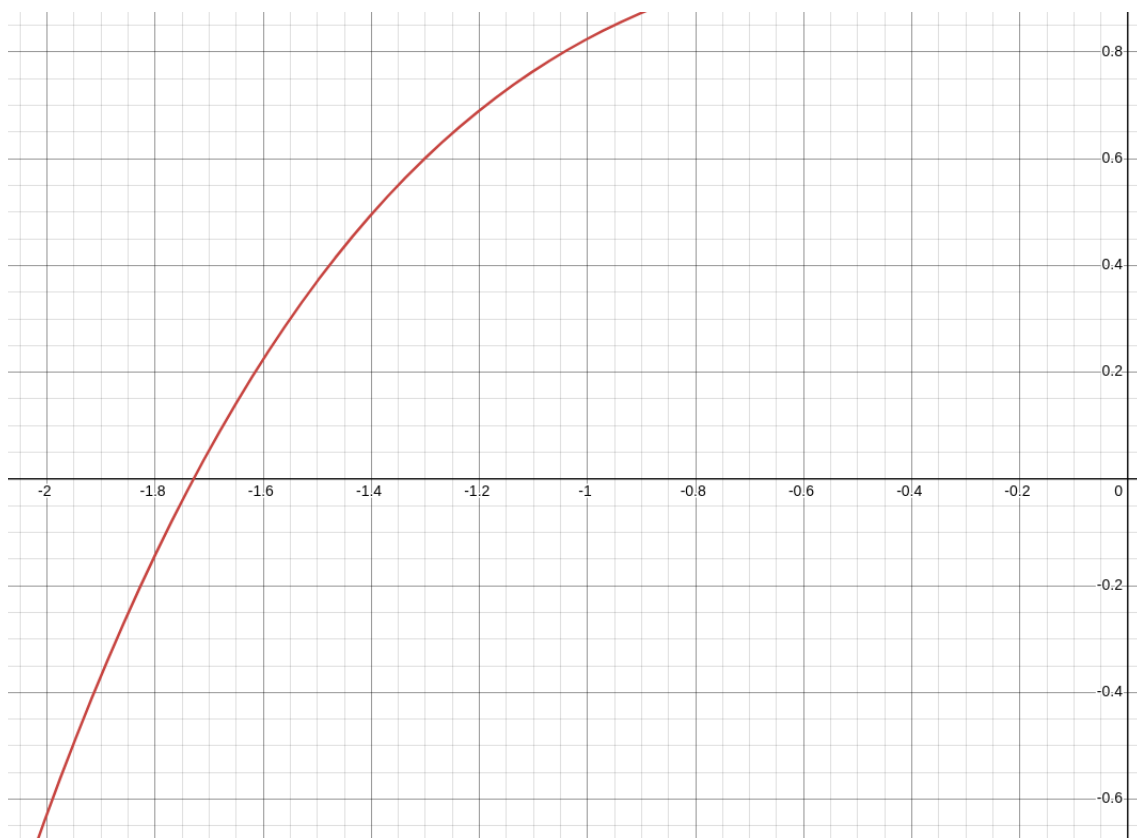


Рис. 1 - График исследуемой функции

Разработка программы.

Программа написана на языке программирования C++.

Для вычисления значения $f(x)$ написана функция *double comp_func(double x)*:

```
double comp_func(double x){  
    return sinh(x)-x+1;  
}
```

Метод бисекции реализован в функции *double Bisect(double left, double right, double eps, int &n)*

Аргументы:

- *double left, double right* – границы исследуемого отрезка;
- *double eps* – точность вычисления корня;
- *int &n* – ссылка на переменную для сохранения кол-ва итераций.

Вычисление округления значений с заданной точностью происходит в функции *double Round(double x, double delta)*

Аргументы:

- *double x* – значения для округления;
- *double delta* – точность округления.

Код программы приведён в Приложении А.

Тестирование.

Точность вычислений	Результат
0.1	-1.6
0.01	-1.73
0.001	-1.729
0.0001	-1.7291
0.00001	-1.72911
0.000001	-1.729116

Исследование зависимости числа итераций от точности вычислений.

На графике (см. рис.2) приведено сравнение полученной при работе программы зависимости количества итераций от заданной точности вычислений с рассчитанными теоретически значениями: $N \approx \log_2 \frac{b-a}{Eps}$. Видно, что практические значения с учётом погрешностей совпадают с теоретическими, следовательно алгоритм реализован корректно.

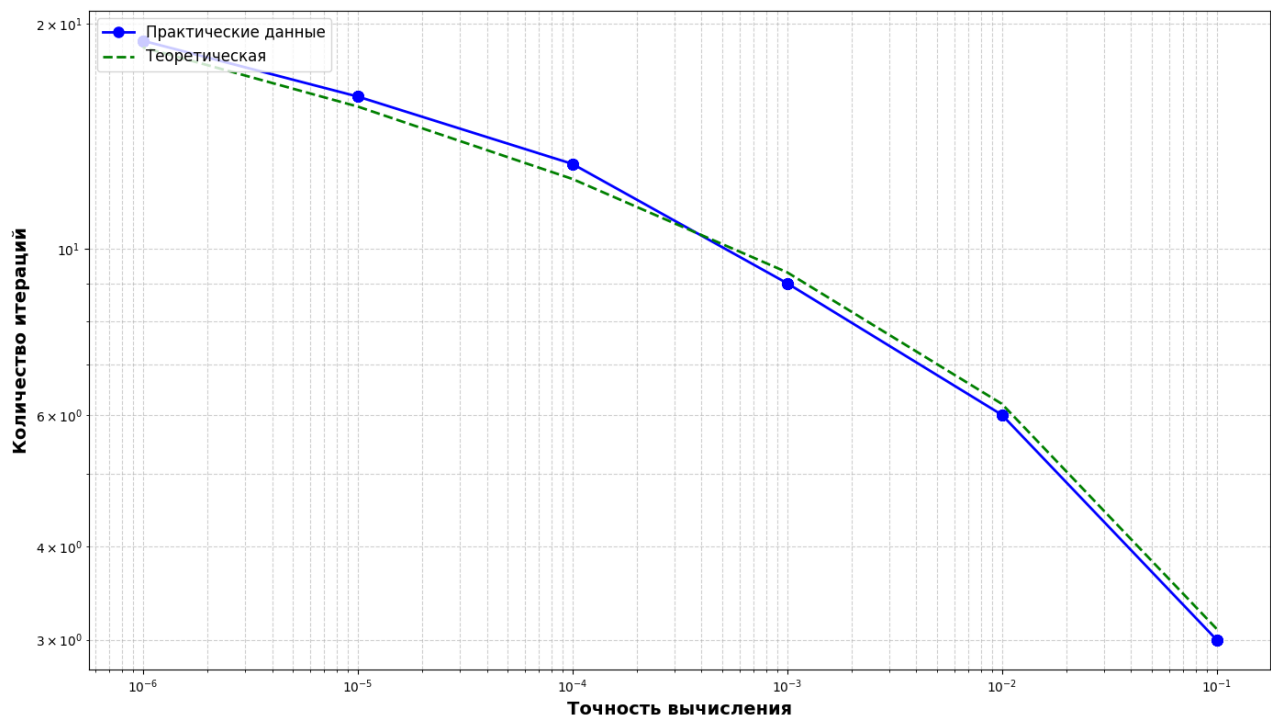


Рис. 2 - Зависимость количества итераций от точности вычислений

Исследование обусловленности метода.

На графике (см. рис.3) приведена зависимость результата вычислений от точности исходных данных, получены несколько кривых, так как для каждой точности исходных данных вычислены результаты с различной точностью.

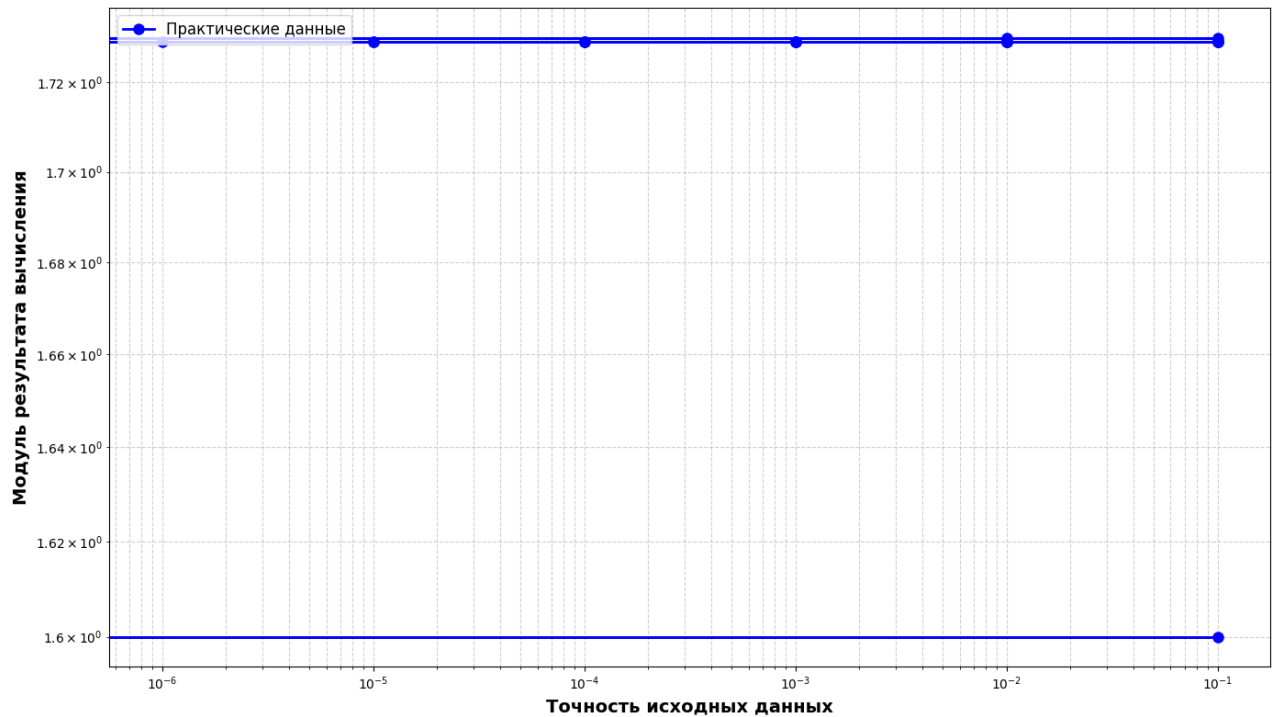


Рис. 3. Зависимость результата вычисления от точности исходных данных

Результат вычислений почти не меняется при выборе различной точности исходных данных, следовательно метод хорошо обусловлен.

Выводы.

В ходе выполнения лабораторной работы был изучен метод бисекции, написана программа с использованием данного метода вычисляющая результат функции с различной точностью. Проведено исследование зависимости количества итераций от точности вычислений, и также исследована обусловленность метода.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
#include <iostream>
#include <fstream>
#include <iomanip>
#include <cmath>

double comp_func(double x){
    return sinh(x)-x+1;
}

double Bisect(double left, double right, double eps, int &n){
    double e=std::abs(eps)*2.0;
    double f_left=comp_func(left), f_right=comp_func(right);
    double x=(left+right)/2.0, y;

    if (f_left*f_right > 0.0){
        std::cout << "Неверное задание интервала\n";
        exit(1);
    }

    if (eps <= 0.0){
        std::cout << "Неверное задание точности\n";
        exit(1);
    }

    n = 0;

    if (f_left == 0.0)
        return left;
    if (f_right == 0.0)
        return right;

    while ((right-left) >= e){
        x = 0.5*(right+left);
        y = comp_func(x);

        if (y == 0.0)
            return (x);

        if (y*f_left < 0.0)
            right = x;
        else{
            left = x;
            f_left = y;
        }
        n++;
    }
    return x;
}

double Round(double x, double delta){
    if (delta <= 1e-9){
        std::cout << "Неверное задание точности округления\n";
        exit(1);
    }
}
```



```

    }

    if (x > 0.0)
        return (delta*(long((x/delta)+0.5)));
    else
        return (delta*(long((x/delta)-0.5)));
}

int main(){
    std::ofstream out("output.txt");

    int fix=1;
    double l=-2.0, r=-1.0;
    for (double eps=0.1; eps >= 0.000001; eps *= 0.1){
        int cnt_it;
        double ans=Bisect(l, r, eps, cnt_it);
        out << std::fixed << std::setprecision(fix) << eps << ' '
<< 0.0 << ' ' << cnt_it << ' ' << ans << '\n';

        for (double delta=0.1; delta > 1e-9; delta *= 0.1){
            ans = Bisect(Round(l, delta), Round(r, delta), eps,
cnt_it);
            out << std::fixed << std::setprecision(fix) << eps << '
' << delta << ' ' << cnt_it << ' ' << std::fixed <<
std::setprecision(fix) << ans << '\n';
        }
        fix++;
    }

    out.close();
}

```